MySQL 的存储引擎有哪些？它们之间有什么区别？ - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新后端、MySQL、数据库面试题，包含详细答案解析和练习题，36242+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

回答重点

MySQL 存储引擎是可插拔的，不同引擎负责数据的存储和读取方式。实际工作中 95% 以上的场景用的都是 **InnoDB**，面试重点把 InnoDB 和 MyISAM 的区别搞清楚就行。

MySQL 8.4 版本一共提供了 10 个引擎，常见的有这几个：

1）InnoDB：MySQL 5.5 之后的默认引擎，支持事务、行级锁、外键，MVCC 也有，适合高并发的 OLTP 场景。数据按聚簇索引组织，主键查询贼快。

2）MyISAM：老版本的默认引擎，不支持事务，只有表级锁，但读性能不错。适合那种写少读多、对一致性要求不高的场景，比如早年的一些报表系统。

3）MEMORY：数据全放内存里，速度快但 MySQL 重启数据就没了。一般拿来做临时表或者会话级缓存。

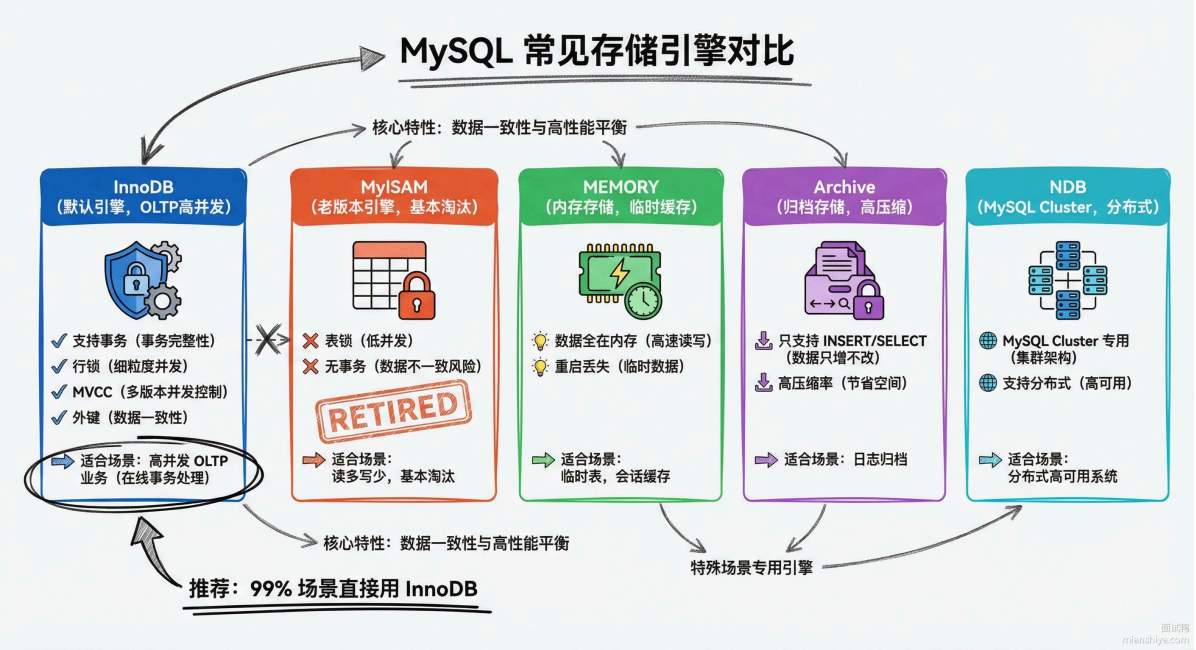
4）Archive：专门存归档数据的，只支持 INSERT 和 SELECT，不支持索引，但压缩率高。日志归档、历史订单这种场景用得上。

5）NDB：MySQL Cluster 用的引擎，支持分布式和高可用，数据自动分片，适合电信级别的大规模集群。

下面这张表列出了各引擎的核心特性对比：

特性	InnoDB	MyISAM	Memory	Archive	NDB
事务支持	Yes	No	No	No	Yes
锁粒度	行锁	表锁	表锁	行锁	行锁
MVCC	Yes	No	No	No	No
外键	Yes	No	No	No	Yes
聚簇索引	Yes	No	No	No	No
B+树索引	Yes	Yes	Yes	No	No
哈希索引	No	No	Yes	No	Yes
全文索引	Yes	Yes	No	No	No
数据压缩	Yes	Yes	No	Yes	No
存储上限	64TB	256TB	受内存限制	无限制	384EB

总结图如下：



扩展知识

InnoDB 和 MyISAM 为什么差别这么大

这俩引擎设计目标就不一样。MyISAM 是 MySQL 早期的引擎，那会儿互联网还没爆发，大部分场景就是读多写少的静态页面，表锁够用了。InnoDB 是后来 Oracle 收购的，从一开始就奔着企业级事务去的，行锁、MVCC、redo log、undo log 这套东西都是为了保证 ACID。

从存储结构看，MyISAM 的索引和数据是分开存的，.MYI 文件存索引，.MYD 文件存数据，查询时先查索引拿到数据行的物理地址，再去数据文件里捞。InnoDB 用的是聚簇索引，数据直接挂在 B+ 树叶子节点上，主键查询一次 B+ 树就能拿到数据，不用二次寻址。

锁机制差别更大。MyISAM 只有表锁，一个写操作会锁住整张表，并发写性能很差。InnoDB 支持行锁，通过 MVCC 实现了读写不阻塞，1000 个并发改同一张表的不同行，互不干扰。

存储引擎怎么选

99% 的业务场景直接用 InnoDB 就完事了。MySQL 5.5 之后 InnoDB 就是默认引擎，官方也一直在优化它，全文索引、空间索引这些原来 MyISAM 有的能力，InnoDB 现在也支持了。

只有几种特殊场景才考虑其他引擎：

1）临时计算结果集，比如 ETL 中间表，可以用 MEMORY 引擎加速，但要注意数据易失性。

2）日志归档、历史流水这种只写不改的数据，Archive 引擎压缩率高，能省不少磁盘。

3）搞 MySQL Cluster 分布式集群，才会用到 NDB。

MyISAM 基本可以淘汰了，MySQL 8.0 已经把系统表从 MyISAM 迁移到 InnoDB 了，官方的态度很明确。

存储引擎版本演进

MySQL 5.5：InnoDB 成为默认引擎，之前是 MyISAM。

MySQL 5.6：InnoDB 支持全文索引，填补了相对 MyISAM 的一个短板。

MySQL 5.7：InnoDB 支持空间索引，JSON 类型也来了。

MySQL 8.0：系统表全部迁移到 InnoDB，MyISAM 只保留用于兼容，新增了原子 DDL。

MySQL 8.4 版本一共提供了 10 个引擎，有兴趣的话可以看看官方文档

> MySQL 8.4 Supported Storage Engines
>
> - `InnoDB`: The default storage engine in MySQL 8.4. `InnoDB` is a transaction-safe (ACID compliant) storage engine for MySQL that has commit, rollback, and crash-recovery capabilities to protect user data. `InnoDB` row-level locking (without escalation to coarser granularity locks) and Oracle-style consistent nonlocking reads increase multi-user concurrency and performance. `InnoDB` stores user data in clustered indexes to reduce I/O for common queries based on primary keys. To maintain data integrity, `InnoDB` also supports `FOREIGN KEY` referential-integrity constraints. For more information about `InnoDB`, see Chapter 17, *The InnoDB Storage Engine*.
> - `MyISAM`: These tables have a small footprint. Table-level locking limits the performance in read/write workloads, so it is often used in read-only or read-mostly workloads in Web and data warehousing configurations.
> - `Memory`: Stores all data in RAM, for fast access in environments that require quick lookups of non-critical data. This engine was formerly known as the `HEAP` engine. Its use cases are decreasing; `InnoDB` with its buffer pool memory area provides a general-purpose and durable way to keep most or all data in memory, and `NDBCLUSTER` provides fast key-value lookups for huge distributed data sets.
> - `CSV`: Its tables are really text files with comma-separated values. CSV tables let you import or dump data in CSV format, to exchange data with scripts and applications that read and write that same format. Because CSV tables are not indexed, you typically keep the data in `InnoDB` tables during normal operation, and only use CSV tables during the import or export stage.
> - `Archive`: These compact, unindexed tables are intended for storing and retrieving large amounts of seldom-referenced historical, archived, or security audit information.
> - `Blackhole`: The Blackhole storage engine accepts but does not store data, similar to the Unix `/dev/null` device. Queries always return an empty set. These tables can be used in replication configurations where DML statements are sent to replica servers, but the source server does not keep its own copy of the data.
> - `NDB` (also known as `NDBCLUSTER`): This clustered database engine is particularly suited for applications that require the highest possible degree of uptime and availability.
> - `Merge`: Enables a MySQL DBA or developer to logically group a series of identical `MyISAM` tables and reference them as one object. Good for VLDB environments such as data warehousing.
> - `Federated`: Offers the ability to link separate MySQL servers to create one logical database from many physical servers. Very good for distributed or data mart environments.
> - `Example`: This engine serves as an example in the MySQL source code that illustrates how to begin writing new storage engines. It is primarily of interest to developers. The storage engine is a "stub" that does nothing. You can create tables with this engine, but no data can be stored in them or retrieved from them.

面试鸭
mianshiya.com

面试官追问

提问：InnoDB 的行锁是怎么实现的？是锁住整行数据吗？

回答：InnoDB 的行锁其实是锁索引记录，不是锁物理行。如果 SQL 走的是主键索引，就锁主键索引上的那条记录；如果走的是二级索引，会先锁二级索引记录，再锁对应的主键记录。如果查询没走索引，那就退化成表锁了，因为要扫全表，所有记录都得锁。所以建索引不光是为了查询快，也是为了锁的粒度更细。

提问：MyISAM 既然只有表锁，为什么还说它读性能好？

> 详细解答可以看：MySQL 中 InnoDB 存储引擎与 MyISAM 存储引擎的区别是什么？

回答：MyISAM 读性能好主要是因为它没有事务开销，不用维护 undo log，也不用做 MVCC 版本链查找。读的时候直接从 B+ 树拿数据地址，去数据文件里顺序读就行。但这个优势是在无并发写或者写很少的前提下，一旦有写操作，表锁会把所有读都阻塞住，性能就崩了。

提问：MEMORY 引擎数据放内存里，和 Redis 比有什么区别？

回答：MEMORY 引擎是 MySQL 进程内的内存存储，只能通过 SQL 访问，数据结构固定是表，重启就丢。Redis 是独立的 KV 存储服务，支持多种数据结构，有持久化机制，可以跨进程访问。如果只是 MySQL 内部做临时表计算，MEMORY 引擎够用；如果是分布式缓存、会话存储这种场景，用 Redis。两者定位完全不一样。

提问：生产环境怎么查看当前表用的是什么存储引擎？

回答：最直接的是 `SHOW TABLE STATUS LIKE 'table_name'`，Engine 列就是引擎类型。也可以 `SHOW CREATE TABLE table_name` 看建表语句最后的 ENGINE 参数。批量查的话用 `SELECT TABLE_NAME, ENGINE FROM information_schema.TABLES WHERE TABLE_SCHEMA='库名'`。